

##

###

6 , . 7 '(Asynchronous)' . setTimeout , (Callback Hell) , Promise . 8 API .

1 (Asynchronous)

- ****: .
- ****: (Single-threaded) , Web API .
- ****: .
- ****: .
- ** **: .

2 (Synchronous) vs

- ****: (Blocking), (Non-blocking).
- ****: () vs () .
- ****: () vs () .
- ****: .
- ** **: blocking non-blocking .

3 (Callback Function)

- ****: .
- ****: .
- ****: setTimeout(() => console.log(''), 1000);
- ****: '' .
- ** **: .

4 (Callback Hell)

- ****: .
- ****: .
- ****: → → → .
- ****: .
- ** **: .

5 Promise

- ****: .
- ****: Pending() → Fulfilled() Rejected() .
- ****: const p = new Promise((resolve, reject) => { if(ok) resolve(''); else reject(''); });
- ****: Promise (.then) (Flat) .
- ** **: Promise 3 .

6 resolve/reject

```
- ****: Promise .
- ****: resolve .then() , reject .catch() .
- ****: reject(' ') catch .
- ****: .
- ** **: .catch() .
```

###

```
| | | |
| :--- | :--- | :--- |
| | (Blocking) | (Non-blocking) |
| | | |
| | | |
```

```
| | (Callback) | (Promise) |
| :--- | :--- | :--- |
| | | .catch() |
| | () | () |
```

###

```
1. : setTimeout 3 ' ' .
2. : Promise setTimeout 2 / .
3. : .then() .
```

TOP 6

```
1. .
2. reject .catch() .
3. Promise resolve/reject .
4. .then() return .
5. / .
6. .catch() .
```

&

```
- ****: "' (Non-blocking)', ''()'!"
- ** **: .
```

###

```
1. [] / .
2. [] setTimeout .
3. [] .
4. [] Promise new Promise() .
5. [] resolve reject .
6. [] .then() .catch() .
```

1. ?

A. .

B. .

C. .

D. .

: D

:

: Web API , .

2. ?

A.

B.

C.

D. setTimeout

: B

:

: " " .

3. Promise 3 ?

A. Start, Process, End

B. Pending, Fulfilled, Rejected

C. Ready, Running, Finished

D. Request, Response, Failed

: B

: Promise

: Promise (Pending), (Fulfilled), (Rejected) 3 .

4. Promise ?

A. .error()

B. .fail()

C. .catch()

D. .reject()

: C

: Promise

: Promise .catch() .

5. resolve ?

A. .then()

B. .finally()

C. .resolved()

D. .wait()

: A

: resolve

: Promise resolve .then() .

: Promise

2 . Promise , .

[]

- Promise
- setTimeout
- resolve reject
- /

[] Promise, setTimeout, resolve, reject

[]

1. 1. / 2 setTimeout
2. 2. resolve(""), reject("")
3. 3. .then() .catch()

[] Promise .then/.catch

[] 100